

Design Goals and Subsystem Decomposition

This document outlines the two key design goals and the breakdown of the ATOM into subsystems.

1.0 Design Goals

ATOM system design is focusing on two essential qualities: usability and efficiency.

1.1 Usability

1.1.1 Usability Goal

Ensuring the design of the ATOM system is intuitive and user-friendly for all roles (Admin, Coordinator, Advisor, Guide, and Guests) by minimizing complexity, creating elegant interfaces and tailoring functionalities to specific roles.

1.1.2 Implementation

- **Simplified Navigation**
 - High cohesion and low coupling.
 - Role-specific dashboards and navigation menus tailored to user permissions (e.g., Admins manage users and events, while Guides see only their events).
 - Labels and menus, such as “View Users” or “Submit Application,” clearly define the purpose of each section.
 - All pages have basic descriptions of what a user can do at the top of the page.
- **Form Design**
 - Forms follow a step-by-step approach, reducing user errors and confusion for guests.
 - Validation ensures inputs like email addresses are correct, improving robustness and fault tolerance.
 - Text boxes for inputs clearly show how the information should be added. (e.g., How many digits of the phone number should be entered)
- **Role-Based Features**
 - Features are displayed dynamically based on the user’s role. This maintains flexibility while simplifying the user experience.
- **Feedback Mechanisms**
 - Visual feedback, such as success messages or actionable buttons, enhances usability by confirming actions and guiding next steps.

1.1.3 Justification

- Simplified workflows reduce the learning curve for new users. Role-specific dashboards simplify navigation for different users.

- Real-time validation saves time and improves efficiency by catching errors early. Visual feedback confirms user actions, enhancing clarity.
- A focus on relevant tasks helps users stay productive without being overwhelmed.
- Step-by-step forms reduce confusion and improve user experience.

1.1.4 Tradeoffs

- Developing role-specific interfaces increases complexity during the development phase.
- Developing user friendly forms also makes for a more complex development phase.
- Simplified features may limit flexibility and functionality for advanced users.

1.2 Efficiency

1.2.1 Efficiency Goal

Designing a system that performs well under various scenarios, such as high traffic or large data loads.

1.2.2 Implementation

- **Automation**
 - Automation of information transfer among tables minimizes the repetitive tasks therefore saves time and increases the overall performance and productivity of the system.
 - All users see relevant data to them at the same speed. Waste of time is minimised when information is passed between one user to another.
- **Optimized Database Design**
 - MongoDB collections are indexed to improve query speed, leveraging persistence principles from system design.
 - Aggregation pipelines handle data processing efficiently, reducing redundancy.
- **Modular APIs**
 - RESTful APIs are designed with clear boundaries (following the subsystem decomposition principle).
 - Batch operations (e.g., assigning guides to multiple events) optimize performance.
- **Frontend Optimization**
 - React's dynamic rendering reduces unnecessary processing.
 - Lazy loading improves page load speeds by only fetching required components.
- **Efficient Scaling**
 - Node.js handles concurrent requests effectively.
 - MongoDB sharding supports horizontal scaling, ensuring consistent performance with growing user and data volumes.

1.2.3 Justification

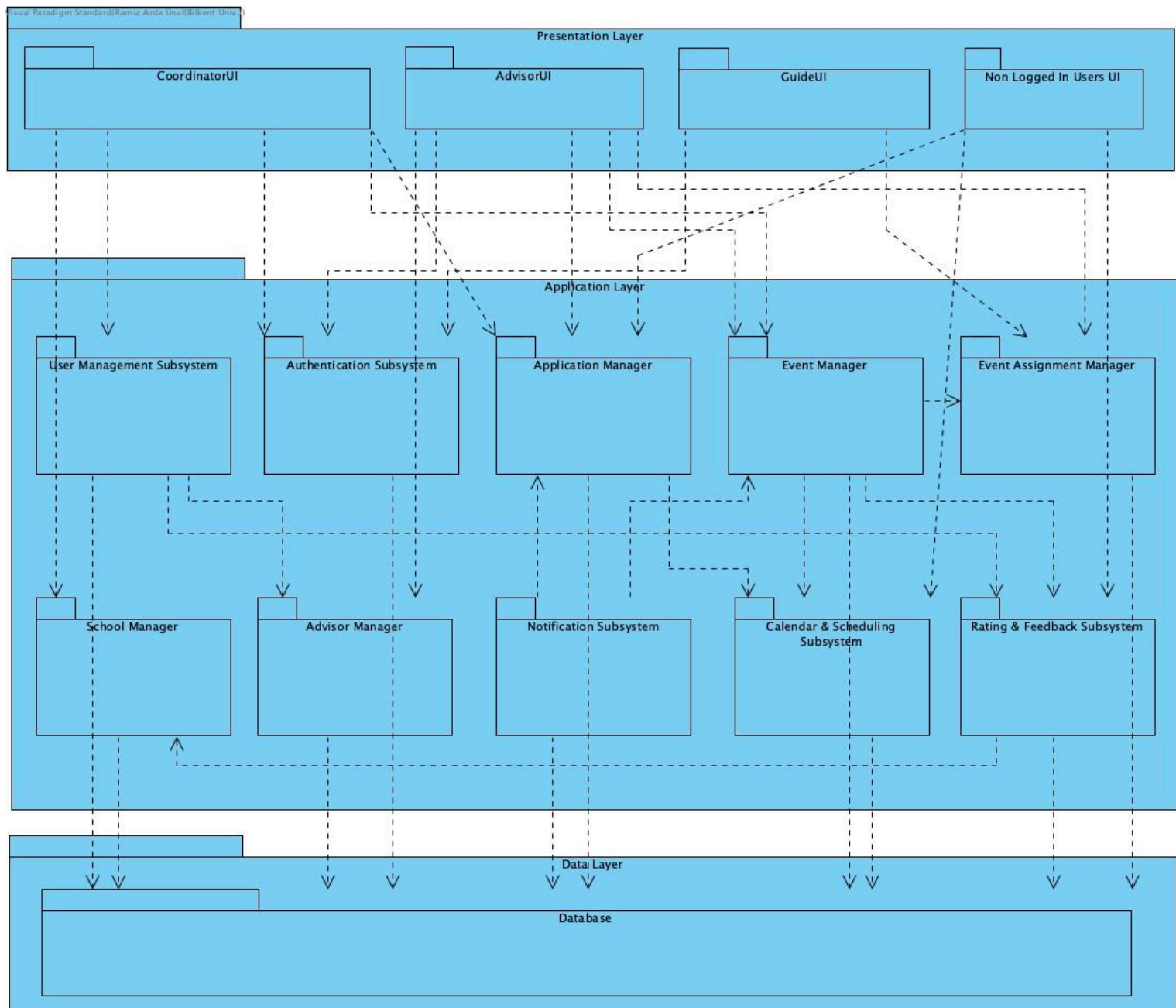
- High performance is achieved through efficient data handling and optimized APIs.
- React's dynamic rendering reduces unnecessary re-renders, enhancing load times.
- Lazy loading ensures only needed components are loaded, reducing initial load time.
- Concurrent requests are handled efficiently, ensuring the system remains responsive under heavy traffic.
- MongoDB sharding supports horizontal scaling, maintaining performance as data grows.

1.2.4 Tradeoffs

- Advanced database optimizations increase initial development time.
- Lazy loading can cause delays for less frequently accessed features.
- Sharding in database may cause complexity in managing distributed data.

2.0 Subsystem Decomposition

The system follows a 3-layer architectural style (Presentation, Application, and Data). This approach increases maintainability ensuring layers to operate independently, enables the addition of new features with minimal impact on other layers thanks to more clear divisions, and allows for easy integration with external systems.



2.1 Presentation Layer

- Uses React.js to dynamically render user-specific dashboards.
- Axios manages efficient communication with the backend.

2.2 Application Layer

- Express.js handles routing, API requests, and business logic.
- Modular services (e.g., user management, event handling) ensure separation of concerns.

2.3 Data Layer

- MongoDB stores all collections with indexed schemas for fast querying.
- Aggregation pipelines enable efficient analytics and reporting.